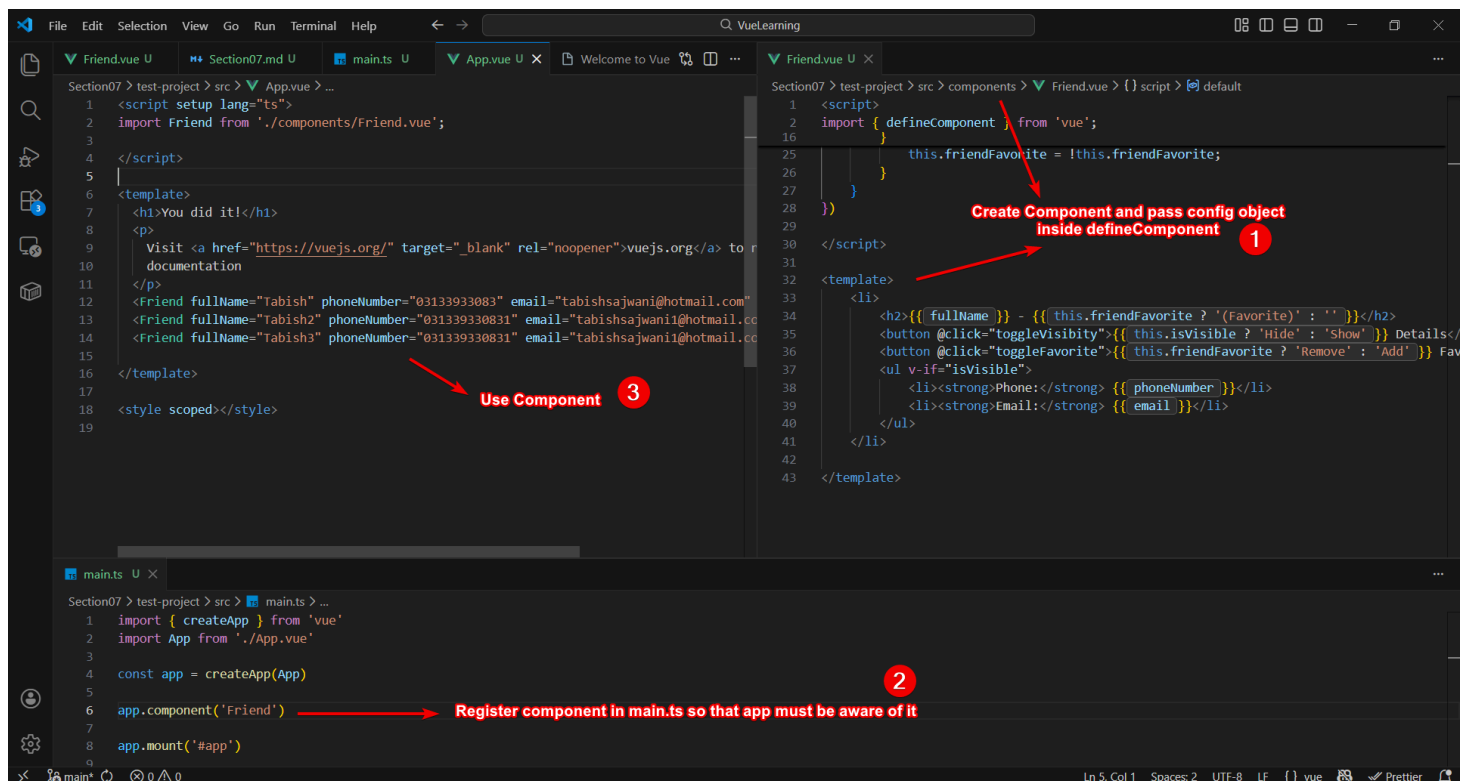


Section 07 - 08 - Components Introduction

Creating Components

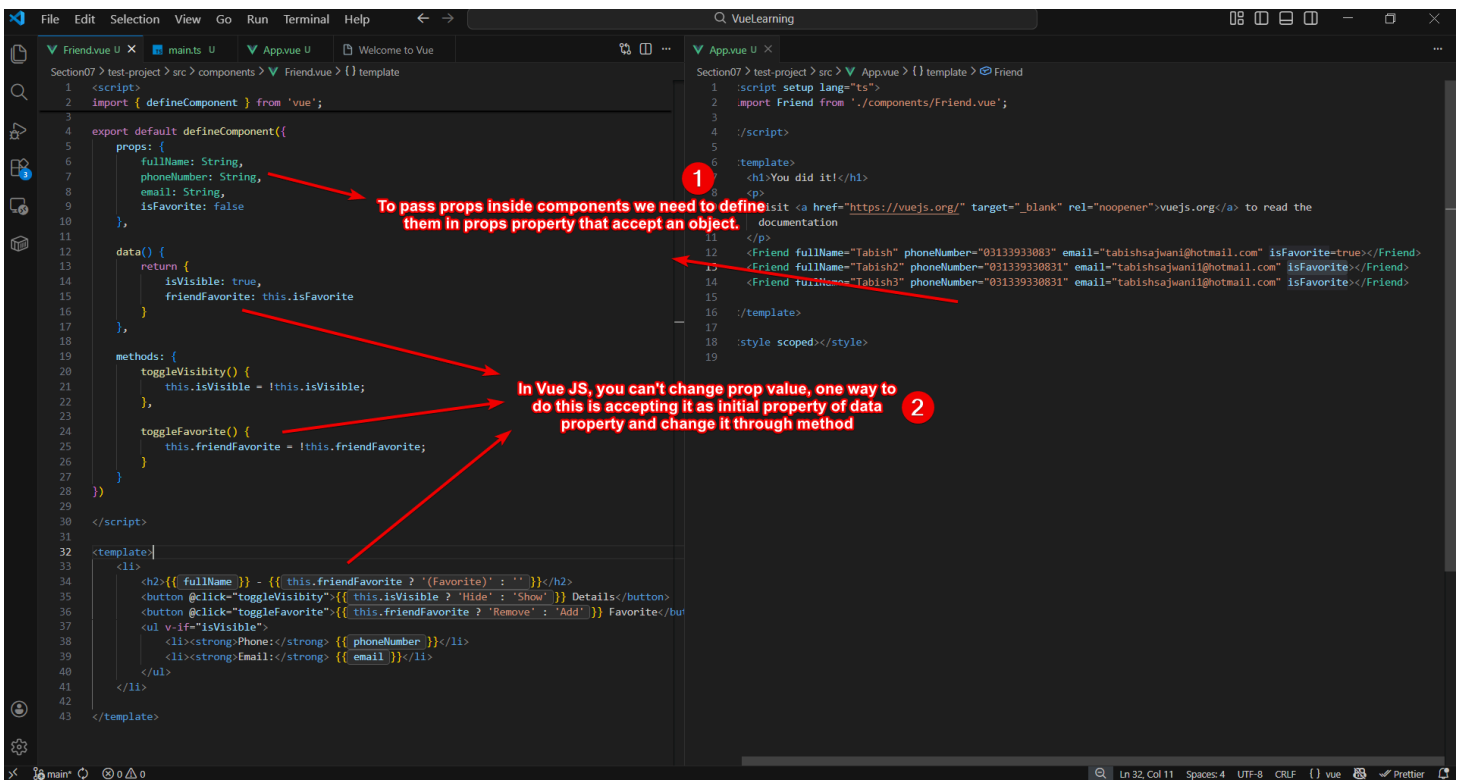
In Vue, you need to create component, register it, and then use it.



Props and Mutating Their Values

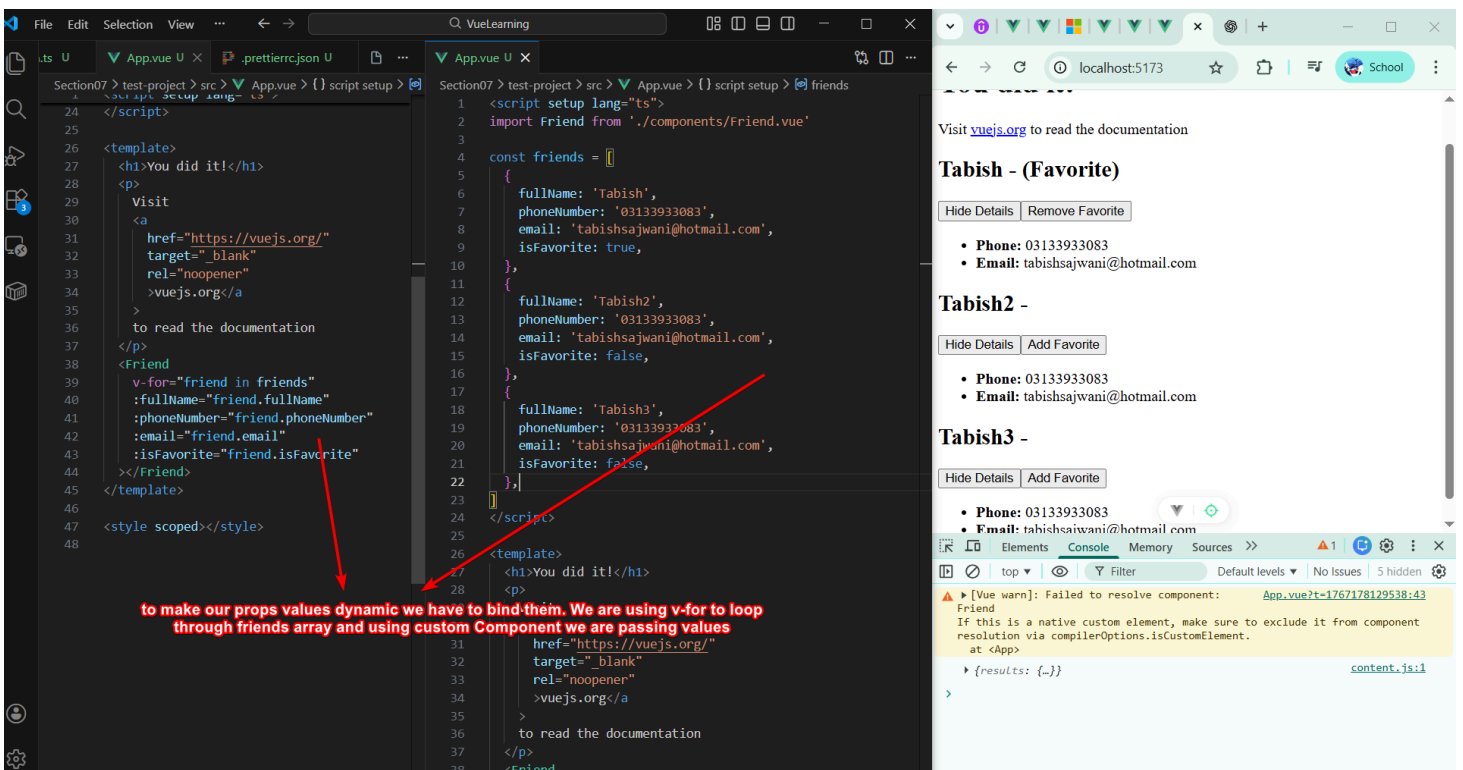
Props are used to **pass data inside components**, but Vue JS **doesn't allow changing their values**.

One way to change prop value is **accepting them as initial state** of one of the data properties and then **changing it through methods**.



Dynamic Props

To make our props values dynamic, we need to bind them using `v-bind`



Child to Parent Values Passing (aka *Lifting the State Up*)

The screenshot shows a code editor with two files: `Friend.vue` and `App.vue`. Red arrows and text annotations explain the process of passing data from child to parent.

Annotation 1: "To pass value from child to parent we need to lift the state up (pass the function), in Vue JS we need to pass custom event. This custom event is pass using build-in variable \$emit that accept event name and any number of arguments." This points to the `toggleFavorite` method in `Friend.vue` which calls `this.$emit('toggleFriendFavorite', this.id)`.

Annotation 2: "we need to pass function in our custom event" This points to the `@toggleFriendFavorite="toggle(friend.id)"` attribute in the `Friend` component within `App.vue`.

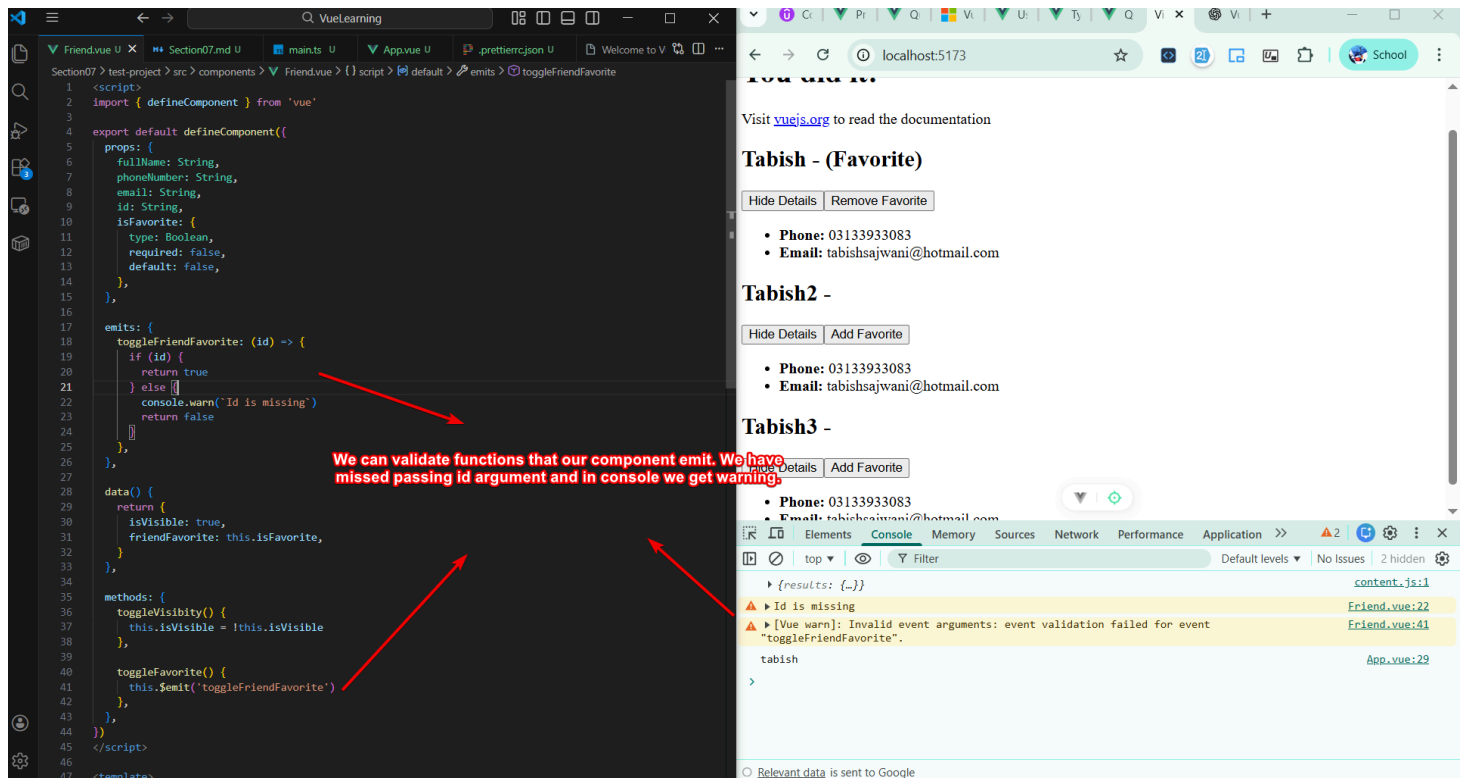
```
1 <script>
4 export default defineComponent({
17
18   data() {
19     return {
20       isVisible: true,
21       friendFavorite: this.isFavorite
22     },
23   },
24   methods: {
25     toggleVisibility() {
26       this.isVisible = !this.isVisible
27     },
28     toggleFavorite() {
29       this.$emit('toggleFriendFavorite', this.id)
30     },
31   },
32 },
33 </script>
34
35 <template>
36 <li>
37   <h2>{{ fullName }} - ({{ this.friendFavorite ? '(Favorite)' : '' }})</h2>
38   <button @click="toggleVisibility">{{ this.isVisible ? 'Hide' : 'Show' }} Details</button>
39   <button @click="toggleFavorite(id)">
40     {{ this.friendFavorite ? 'Remove' : 'Add' }} Favorite
41   </button>
42   <ul v-if="isVisible">
43     <li><strong>Phone:</strong> {{ phoneNumber }}</li>
44     <li><strong>Email:</strong> {{ email }}</li>
45   </ul>
46 </li>
47 </template>
48
49
```

```
1 <script setup lang="ts">
4 const { toggle } = useToggle()
26
27
28 const toggle = (id: string) => {
29   console.log(id)
30 }
31
32 <template>
33 <h1>You did it!</h1>
34 <p>
35   Visit
36   <a
37     href="https://vuejs.org/"
38     target="_blank"
39     rel="noopener"
40     >vuejs.org</a>
41   </a>
42   to read the documentation
43 </p>
44 <Friend
45   v-for="friend in friends"
46   :key="friend.id"
47   :fullName="friend.fullName"
48   :phoneNumber="friend.phoneNumber"
49   :email="friend.email"
50   :isFavorite="friend.isFavorite"
51   @toggleFriendFavorite="toggle(friend.id)"
52 ></Friend>
53 </template>
54
55 <style scoped></style>
56
57
```

To pass value from **child to parent**, we need to **create custom event** using `$emit` built-in method that accepts **custom event name** and any amount of argument.

if your function emitting a custom event, you need to pass `emits` property in config object that accepts a string of array or an object containing name of custom event and validation.

Emit Function Validation



We can validate emit function.

Provide / Inject - *Equiv. useContext*

Vue allows **Provide** and **Inject** properties that is similar to `createContext` and `useContext` in React.

In parent app (**App.vue**), you can pass `provide` key in config object that accepts a function that should return an object containing key as variable and value as data.

Then in child app (**UserData.vue**) to can use this provided data, using `inject` property. Inject property accepts an array of strings containing name of keys that is provided in `provide` object.

Provide/inject works only in parent/child relationship, it will not work in passing data to neighbour/sibling component.

Only use provide/inject only if you have pass-through component.

Summary

Component Communication

Component Communication Overview

Components are used to build UIs by **combining** them

Components build “**parent-child**” relations and use “**unidirectional** data flows” for communication

Custom Events (child => parent)

“**Custom Events**” are **emitted** (via `$emit`) to trigger a method in a parent component

Custom events can **carry data** which can be used in the called method

Props (parent => child)

“**Props**” are used to **pass data from a parent to a child** component

Props should be **defined in advance**, possibly in great detail (type, required etc)

Provide-Inject

If data needs to be passed **across multiple components** (“pass-through”), you can use **provide/ inject**

Provide data in a parent component, **inject it into a child** component